

CaffeinatedData

Customer Sentiment Analysis with Neural Networks Project

IMDB + Yelp + Amazon dataset

Research Question:

“Can a neural network model accurately classify the sentiment of a customers review as positive, negative or neutral based on review text?”

Project Goal:

The objective is to build and evaluate a neural network model that predicts whether a review is positive or negative based on review text (binary classification or essentially a logistic regression (Sewell, n.d.)). The dataset used will be the IMDB, Yelp, and Amazon reviews with a binary sentiment(0 = negative sentiment, 1 = positive sentiment).

Model Justification:

A Long Short-Term Memory (LSTM) recurrent neural network (RNN) was selected for this text classification task. Unlike traditional feedforward neural networks, LSTMs are designed to process sequences of text and maintain information from earlier words in a review (Sewell, n.d.). This enables the model to learn contextual relationships and sentiment patterns within the review text, making it appropriate for predicting whether a review expresses positive or negative sentiment.

Data Preparation:

Exploratory Data Analysis:

Combining the three datasets (Imdb, Yelp and Amazon) yielded 3000 rows, half being positive sentiment and the other half being negative sentiment which pushes this neural network to being a binary classification as there is no neutral sentiment.

Dataset summary prior to processing:

In the screenshot below, we can see the number of reviews is 3000, with 1500 positive and 1500 negative sentiments, the total words are 35495, with characters totaling 195814, and a vocabulary size of 8015 unique words.

```
# Dataset summary before creating total_words and total_chars and cleaning data

print("Number of Reviews:", len(df))

print("Positive Reviews:", (df["sentiment"] == 1).sum())
print("Negative Reviews:", (df["sentiment"] == 0).sum())

raw_words = df["review"].apply(lambda x: len(str(x).split())).sum()
print("Total Words:", raw_words)

raw_chars = df["review"].apply(lambda x: len(str(x))).sum()
print("Total Characters:", raw_chars)

raw_vocab = len(set(" ".join(df["review"].astype(str)).split()))
print("Vocabulary Size:", raw_vocab)
```

```
Number of Reviews: 3000
Positive Reviews: 1500
Negative Reviews: 1500
Total Words: 35495
Total Characters: 195814
Vocabulary Size: 8015
```

Word clouds for positive sentiment after processing:

As we can see, words like food, great, place, good, phone, movie, film are highly prevalent.

English letters, special symbols and other Unicode characters. The search identified 17 reviews containing unusual characters. Examples included boxes as shown in the screenshot below. As shown, there are unusual box characters.

```
[39]: # Find rows containing unusual characters

#Using contains with ^ to ask for NOT IN, \x00-\x7f (ASCII characters which is the Letters, 0-9 and standard punctuation (which will be cleared out in fu

unicode_rows = df[df["review"].str.contains(r"[^\x00-\x7F]", regex=True, na=False)]

print("Rows containing unusual characters:", len(unicode_rows))

unicode_rows.head()

Rows containing unusual characters: 17
[39]:
```

	index	review	sentiment
18	18	It's practically perfect in all of them □ a tr...	1
80	80	I'm glad this pretentious piece of s*** didn't...	0
178	178	The script is□was there a script?	0
182	182	I'll even say it again □ this is torture.	0
424	424	This show is made for Americans - it is too st...	0

Vocabulary size:

Vocabulary size was calculated by counting the number of unique words in the dataset. Before preprocessing, the dataset contained 8,015 unique words. After preprocessing, the vocabulary size decreased to 4,648 unique words due to the removal of stop words (stopwords from the **nlTK** library, numbers, punctuation, loose characters from the removal of punctuation and other non-informative text)

Before data cleaning:

```
[37]: # Dataset summary before creating total_words and total_chars and cleaning data

print("Number of Reviews:", len(df))

print("Positive Reviews:", (df["sentiment"] == 1).sum())
print("Negative Reviews:", (df["sentiment"] == 0).sum())

raw_words = df["review"].apply(lambda x: len(str(x).split())).sum()
print("Total Words:", raw_words)

raw_chars = df["review"].apply(lambda x: len(str(x))).sum()
print("Total Characters:", raw_chars)

raw_vocab = len(set(" ".join(df["review"].astype(str)).split()))
print("Vocabulary Size:", raw_vocab)

Number of Reviews: 3000
Positive Reviews: 1500
Negative Reviews: 1500
Total Words: 35495
Total Characters: 195814
Vocabulary Size: 8015
```

After data cleaning:

```
# Dataset summary after cleaning

print("Number of Reviews:", len(df))

print("Positive Reviews:", (df["sentiment"] == 1).sum())
print("Negative Reviews:", (df["sentiment"] == 0).sum())

print("Total Words:", df["total_words"].sum())
print("Total Characters:", df["total_chars"].sum())

clean_vocab = len(set(" ".join(df["review"]).split()))
print("Vocabulary Size:", clean_vocab)
```

```
Number of Reviews: 2937
Positive Reviews: 1472
Negative Reviews: 1465
Total Words: 18765
Total Characters: 121150
Vocabulary Size: 4648
```

Proposed word embedding length:

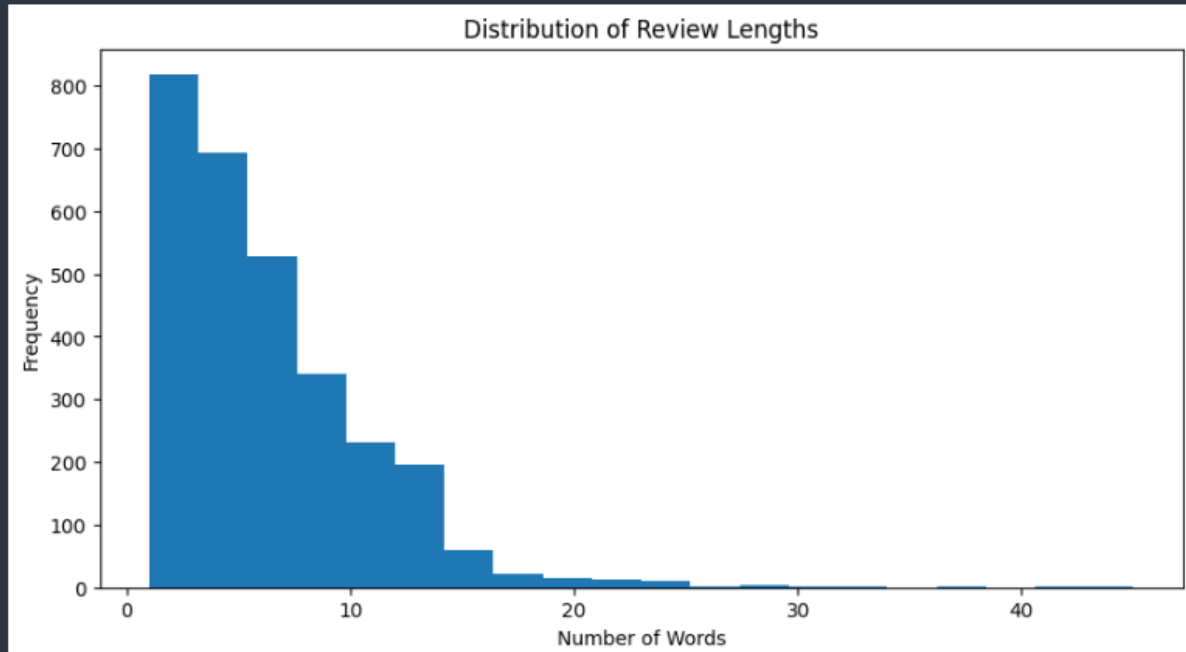
An embedding dimension of 128 was initially selected based on instructor guidance (Sewell, n.d.). Model performance was then evaluated to determine whether a smaller embedding dimension would provide comparable accuracy while reducing model complexity. If the model overfits, the embedding length will be reduced to 64.

Statistical justification for the chosen maximum sequence length:

Review lengths were analyzed prior to model development using the interquartile range (IQR) method. The first quartile (Q1) was 3 words and the third quartile (Q3) was 8 words, resulting in an upper fence of 15.5 words. Reviews exceeding 16 words were considered length outliers and removed from the dataset. This process removed 95 reviews (3.23% of observations). A maximum sequence length of 16 words was subsequently selected to represent the majority of reviews while reducing excessive padding and model complexity.

	total_words																total_chars	
	count	mean	std	min	25%	50%	75%	max	count	mean	std	min	25%	50%	75%	max		
sentiment																		
0	1465.0	6.255290	3.989443	1.0	3.0	5.0	8.0	32.0	1465.0	40.076451	26.658889	2.0	21.0	34.0	53.0	215.0		
1	1472.0	6.522418	4.545507	1.0	3.0	5.0	8.0	45.0	1472.0	42.417120	31.600926	4.0	21.0	34.0	55.0	349.0		

```
: #Histogram of the sentence length  
  
import matplotlib.pyplot as plt  
  
plt.figure(figsize=(10,5))  
plt.hist(df["total_words"], bins=20)  
plt.title("Distribution of Review Lengths")  
plt.xlabel("Number of Words")  
plt.ylabel("Frequency")  
plt.show()
```



As we can see in the distribution histogram, it's a unimodal far right skew due to outliers with far more than average word count.

[281]: *#Checking to see what is the upper quartile word count for sentences (Aka anything past is an outlier)*

```
q1 = df["total_words"].quantile(0.25)
q3 = df["total_words"].quantile(0.75)
```

```
iqr = q3 - q1
```

```
upper_fence = q3 + 1.5 * iqr
```

```
print(upper_fence)
```

15.5

Dr. Sewell pretty much says that maximum sentence length isn't a law but a hyperparameter that is debated.

[290]:

```
q1 = df["total_words"].quantile(0.25)
q3 = df["total_words"].quantile(0.75)
```

```
print(q1)
```

```
print(q3)
```

```
upper_fence = q3 + 1.5 * iqr
```

```
removed = (df["total_words"] > upper_fence).sum()
```

```
print("Reviews removed:", removed)
```

```
print("Percent removed:", round(removed / len(df) * 100, 2), "%")
```

3.0

8.0

Reviews removed: 95

Percent removed: 3.23 %

Goals of Tokenization for Data Preparation:

The goal of tokenization is to convert text reviews into numerical representations that can be processed by a neural network. Because neural networks cannot directly interpret words, each unique word is assigned a numerical index based on a generated vocabulary.

List of packages used prior to tokenization:

```
from nltk.stem import WordNetLemmatizer
```

```
import nltk
```

Code for normalizing text prior to tokenization:

After combining the three data sets together, the **nltk stopwords and lemmatizer are downloaded for future processing.**

```
#download the stop words

nltk.download('stopwords')

#print stop words
|
stop_words = (nltk.corpus.stopwords.words('english'))

[nltk_data] Downloading package stopwords to
[nltk_data]   C:\Users\nam\AppData\Roaming\nltk_data...
[nltk_data]   Package stopwords is already up-to-date!

#Download the lemmatizer from nltk
#Lemmatizer is to reduce the word to its base form, requirement of Dr. Sewell and Dr. Aly
nltk.download("wordnet")
nltk.download("omw-1.4")
```

The first step was to replace accented letters such as **é** to **e**, this ensures that the accented letters would not be removed from future code when it comes to removing punctuation and at the same time preserving the word. This won't hurt the words interpretation by the machine learning model while still being easily interpretable by humans

```
#Swap unicode, basically change the accented letters to non-accented
```

```
replacement_dict = {  
    "é": "e",  
    "è": "e",  
    "ê": "e",  
    "ë": "e",  
    "á": "a",  
    "à": "a",  
    "â": "a",  
    "ä": "a",  
    "í": "i",  
    "ì": "i",  
    "î": "i",  
    "ï": "i",  
    "ó": "o",  
    "ò": "o",  
    "ô": "o",  
    "ö": "o",  
    "ú": "u",  
    "ù": "u",  
    "û": "u",  
    "ç": "c",  
    "ñ": "n",  
    "ã": "a"  
}  
  
for old, new in replacement_dict.items():  
    df["review"] = df["review"].str.replace(old, new, regex=False)
```

The data is now then transformed into lower case

```
#Prepare the dataset  
#Convert all to lower string  
#Replace punctuations with space using regex  
  
df["review"] = df["review"].str.lower()
```

The data is now stripped off of the stop words from the **nlTK** library that was previously downloaded.

This was processed this way as if the punctuation was removed prior to removing stop words, it would be more difficult to remove stranded characters as for example, don't would have became don t. The lambda code would have not catch that.

```
: #Keep only non stop words

df["review"] = df["review"].apply(
    lambda x: " ".join(
        word for word in x.split()
        if word not in stop_words
    )
)
```

Now the punctuation is stripped from the data

```
[47]: #Brackets = match one character from this group
# ^ = NOT
#\w = word characters
#\s white space
#basically means match any character that is not a word character or whitespace

df["review"] = df["review"].str.replace(
    r"[^\w\s]",
    " ",
    regex=True
)
```

Now the whitespaces are stripped and normalized, as the punctuations were replaced with a space.

```
: #Strip the white spaces at the beginning and end of sentences just in case

df["review"] = df["review"].str.strip()
```

There are some duplicate reviews, most consisting of one word reviews, those were removed.

```
#check what rows are duplicated
```

```
duplicates = df[df.duplicated()]
```

```
print(duplicates.head(20))
```

	review	sentiment
235	horrible	0
257	funny	1
363	definitely worth checking out	1
418	movie bad	0
585	recommended	0
615	funny	1
766	boring	0
788	10 10	1
801	avoid costs	0
837	waste time	0
867	great movie	1
886	avoid costs	0
1143	disappointed	0
1146	waste time	0
1262	works great	1
1285	great phone	1
1290	great phone	1
1300	sending back	0
1337	disappointing	0
1366	waste money	0

```
#Removing duplicates
```

```
print(df.shape)
```

```
df = df.drop_duplicates()
```

```
print(df.shape)
```

```
print(df.duplicated().sum())
```

```
(3000, 2)
```

```
(2939, 2)
```

```
0
```

Double spaces are removed, data is restriped once more to remove spaces from the beginning and end of strings.

```
#There are some double spaces. replace spaces that are double and more with a single space
```

```
df["review"] = df["review"].str.replace(r"\s+", " ", regex=True)
```

```
#Restrip because replacing could leave edge cases
```

```
df["review"] = df["review"].str.strip()
```

Double checking to see if data is cleaned:

```
6]: #Check for @
    at_rows = df["review"].str.contains("@", na=False)
    print(len(at_rows))

0

7]: #Check for new lines

df["review"].str.contains("\n", regex=False).sum()

7]: 0
```

```
: #Checking to see special characters are removed

accented_chars = set()

for text in df["review"]:
    for char in text:
        if ord(char) > 127:
            accented_chars.add(char)

print(sorted(accented_chars))

#They are removed

[]

Though the dataset isn't exactly 1500 positive and 1500 negative because of duplicate reviews But it's a very small difference that shouldn't influence bias
```

Removing numbers, tabs, normalizing whitespace for good measure.

```
: #Remove all digits
#Normalize all white space

#\d is all digits, + means more than one
df["review"] = df["review"].str.replace(r"\d+", "", regex=True)

#\s is white space, tabs, etc
df["review"] = df["review"].str.replace(r"\s+", " ", regex=True)
df["review"] = df["review"].str.strip()
```

Removing standalone s, as removing punctuation has left standalone possessive 's' (ex: Joe's > joe s)

```
: #Removing standalone 's'

# \b means boundary, so two of them sandwiching the s ensures that it deletes only 's' that has a space preceding and after
df["review"] = df["review"].str.replace(r"\bs\b", "", regex=True)

# Clean up extra space
#stripping space from beginning and end

df["review"] = df["review"].str.replace(r"s+", " ", regex=True)
df["review"] = df["review"].str.strip()

: #double checking for standalone s

df["review"].str.contains(r"\bs\b", regex=True, na=False).sum()

print(df[df["review"].str.contains(r"\bs\b", regex=True, na=False)].head(20))

Empty DataFrame
Columns: [review, sentiment, total_words, total_chars]
Index: []
```

Words are then **lemmatized()** with **WordNetLemmatizer()**

Lemmatize was chosen instead of stemming as it is much more interpretable(Aly, n.d.).

```
#Lemmatize the words

lemmatizer = WordNetLemmatizer()

df["review"] = df["review"].apply(
    lambda x: " ".join(
        lemmatizer.lemmatize(word)
        for word in x.split()
    )
)
```

Data is then checked to verify all characters

```
# Identify characters (Elleh, n.d.)
character_search = df['review']

# Use nested loop to identify all characters
# SOURCE --- (Elleh, n.d.)
character_list = []

for review in character_search:
    for character in review:
        if character not in character_list:
            character_list.append(character)
            print(character_list)

['v']
['v', 'e']
['v', 'e', 'r']
['v', 'e', 'r', 'y']
['v', 'e', 'r', 'y', ' ']
['v', 'e', 'r', 'y', ' ', 's']
['v', 'e', 'r', 'y', ' ', 's', 'l']
['v', 'e', 'r', 'y', ' ', 's', 'l', 'o']
['v', 'e', 'r', 'y', ' ', 's', 'l', 'o', 'w']
['v', 'e', 'r', 'y', ' ', 's', 'l', 'o', 'w', 'm']
['v', 'e', 'r', 'y', ' ', 's', 'l', 'o', 'w', 'm', 'i']
['v', 'e', 'r', 'y', ' ', 's', 'l', 'o', 'w', 'm', 'i', 'n']
['v', 'e', 'r', 'y', ' ', 's', 'l', 'o', 'w', 'm', 'i', 'n', 'g']
['v', 'e', 'r', 'y', ' ', 's', 'l', 'o', 'w', 'm', 'i', 'n', 'g', 'a']
['v', 'e', 'r', 'y', ' ', 's', 'l', 'o', 'w', 'm', 'i', 'n', 'g', 'a', 'd']
['v', 'e', 'r', 'y', ' ', 's', 'l', 'o', 'w', 'm', 'i', 'n', 'g', 'a', 'd', 't']
['v', 'e', 'r', 'y', ' ', 's', 'l', 'o', 'w', 'm', 'i', 'n', 'g', 'a', 'd', 't', 'f']
['v', 'e', 'r', 'y', ' ', 's', 'l', 'o', 'w', 'm', 'i', 'n', 'g', 'a', 'd', 't', 'f', 'u']
['v', 'e', 'r', 'y', ' ', 's', 'l', 'o', 'w', 'm', 'i', 'n', 'g', 'a', 'd', 't', 'f', 'u', 'c']
['v', 'e', 'r', 'y', ' ', 's', 'l', 'o', 'w', 'm', 'i', 'n', 'g', 'a', 'd', 't', 'f', 'u', 'c', 'h']
['v', 'e', 'r', 'y', ' ', 's', 'l', 'o', 'w', 'm', 'i', 'n', 'g', 'a', 'd', 't', 'f', 'u', 'c', 'h', 'k']
['v', 'e', 'r', 'y', ' ', 's', 'l', 'o', 'w', 'm', 'i', 'n', 'g', 'a', 'd', 't', 'f', 'u', 'c', 'h', 'k', 'p']
['v', 'e', 'r', 'y', ' ', 's', 'l', 'o', 'w', 'm', 'i', 'n', 'g', 'a', 'd', 't', 'f', 'u', 'c', 'h', 'k', 'p', 'b']
['v', 'e', 'r', 'y', ' ', 's', 'l', 'o', 'w', 'm', 'i', 'n', 'g', 'a', 'd', 't', 'f', 'u', 'c', 'h', 'k', 'p', 'b', 'x']
['v', 'e', 'r', 'y', ' ', 's', 'l', 'o', 'w', 'm', 'i', 'n', 'g', 'a', 'd', 't', 'f', 'u', 'c', 'h', 'k', 'p', 'b', 'x', 'j']
['v', 'e', 'r', 'y', ' ', 's', 'l', 'o', 'w', 'm', 'i', 'n', 'g', 'a', 'd', 't', 'f', 'u', 'c', 'h', 'k', 'p', 'b', 'x', 'j', 'z']
['v', 'e', 'r', 'y', ' ', 's', 'l', 'o', 'w', 'm', 'i', 'n', 'g', 'a', 'd', 't', 'f', 'u', 'c', 'h', 'k', 'p', 'b', 'x', 'j', 'z', 'q']
```

As we can see the last line contains all 26 alphabets and a space. This means that the data is fully processed for tokenization.

Outliers are then removed with accordance to the justified **chosen maximum sequence length**.

This code filters for reviews that are 16 words in length or less.

```
: #Settling on 16 words as max for sentence length. Must filter the data to only have 16 words max

# Remove length outliers using IQR upper fence
df = df[df["total_words"] <= 16].copy()

print(df.shape)

(2866, 4)
```

Hyper parameters were taken note of

List of Hyperparameters needed for tokenization:

Max_sequence_length = 16.

Padding = 'post', this ensures that the LSTM layer immediately sees real words instead of 0's to process first.

Truncating = 'post'

Tokenization process:

A tokenizer was created using Kera's Tokenizer() class and fitted to the cleaned review text. The tokenizer scanned all reviews and generated a vocabulary by assigning a unique integer index to each word based on its frequency within the dataset. The text_to_sequences() function was then used to convert each review from text into a sequence of integer (each integer matching the index of the word in the vocabulary). This transformation converted the text data into a numerical format that could be processed by the neural network as neural networks cannot interpret word, hence they were numerically transformed.

To verify that tokenization was successful, a sample of the generated sequences was compared to the original review text.

```

# Create a tokenizer to build a vocabulary from the cleaned review text
tokenizer = Tokenizer()
tokenizer.fit_on_texts(df['review'])

# Convert text reviews into integer sequences, basically Vectorize

sequences = tokenizer.texts_to_sequences(df["review"])

df['sequence'] = sequences
print(df['sequence'].head(10))
print(df['review'].head(10))

0      [355, 355, 147, 611, 1909, 2, 1910, 1911, 1247...
1      [188, 402, 612, 39, 728, 729, 357, 537, 290]
3      [63, 175, 234, 917, 453]
4      [18, 78, 2, 1912, 403, 148, 454, 291, 538, 539]
5      [455, 2, 540, 235, 456, 730, 1913, 14, 358, 613]
6      [359, 59, 104]
7      [257, 2, 360, 128, 1, 731, 1, 918, 292]
8      [199, 320]
9      [134, 614, 1914, 236, 1915, 1916]
10     [919, 1917, 732]
Name: sequence, dtype: object
0      very very slow moving aimless movie distressed...
1      sure lost flat character audience nearly half ...
3      little music anything speak of
4      best scene movie gerardo trying find song keep...
5      rest movie lack art charm meaning emptiness wo...
6      wasted two hour
7      saw movie today thought good effort good messa...
8      bit predictable
9      loved casting jimmy buffet science teacher
10     baby owl adorable
Name: review, dtype: str

```

Padding Process:

After tokenization, the review sequences contained different numbers of tokens because reviews varied in length. Neural networks require inputs to have the same dimensions, so sequence padding was applied using TensorFlow's `pad_sequences()` function. A maximum sequence length of 16 words was selected and justified in question B1, based on the IQR analysis of review lengths (Sewell, n.d.). Reviews shorter than 16 words were padded with zeros at the end of the sequence (`padding='post'`), while any review longer than 16 words would be truncated from the end (`truncating='post'`). Post-padding was selected to preserve the beginning of each review and maintain a consistent input shape for the LSTM model. Post-padding achieves lower accuracy compared to pre-padding, the early pre 0's add noise to the sequence (Reddy & Reddy, 2019)

Sequence example for elaboration:

Example of tokenization: the review "baby owl adorable" was converted into the numerical sequence [919, 1917, 732], where each integer represents a unique word in the vocabulary.

```
0      [355, 355, 147, 611, 1909, 2, 1910, 1911, 1247...
1      [188, 402, 612, 39, 728, 729, 357, 537, 290]
3      [63, 175, 234, 917, 453]
4      [18, 78, 2, 1912, 403, 148, 454, 291, 538, 539]
5      [455, 2, 540, 235, 456, 730, 1913, 14, 358, 613]
6      [359, 59, 104]
7      [257, 2, 360, 128, 1, 731, 1, 918, 292]
8      [199, 320]
9      [134, 614, 1914, 236, 1915, 1916]
10     [919, 1917, 732]
Name: sequence, dtype: object
0      very very slow moving aimless movie distressed...
1      sure lost flat character audience nearly half ...
3      little music anything speak of
4      best scene movie gerardo trying find song keep...
5      rest movie lack art charm meaning emptiness wo...
6      wasted two hour
7      saw movie today thought good effort good messa...
8      bit predictable
9      loved casting jimmy buffet science teacher
10     baby owl adorable
Name: review, dtype: str
```

Screenshot of padded Sequence:

Example of a padded sequence. Zeros were appended to the end of the tokenized review to achieve the maximum sequence length of 16.

```
10     [919, 1917, 732, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ...
```

10

baby owl adorable

Categories of Sentiment:

Two sentiment categories will be used in this model: positive (1) and negative (0). Because the target variable is binary, the final dense layer will contain one neuron with a sigmoid activation function (Sewell, n.d.). The sigmoid function outputs a probability value between 0 and 1, allowing the model to classify each review as either positive or negative sentiment.

Data Preparation Step Summary:

Most of the data preparation steps were previously described in questions B1-B4. A summary of the process is provided below:

1. Import the Imdb, Yelp, and Amazon review datasets into separate Pandas Dataframe
2. Combine the three datasets into a single dataframe using the `concat()` function.
3. Checked for missing values, duplicate records, and unusual characters
4. Perform exploratory data analysis by examining the sentiment distribution, vocabulary size, review lengths and summary statistics
5. Normalized the text by converting all characters to lowercase.
6. Replaced accented characters with their non-accented equivalents to improve consistency (this is to setup for filters to ensure the data will be normalized to ASCII standard)
7. Removed stop words using the NLTK stopwords library.
8. Removed punctuation and special characters using regular expressions.
9. Tokenized the review text by converting words into integer sequences using Keras Tokenizer.
10. Analyzed review lengths using summary statistics and the IQR method to identify unusually long reviews and determine an appropriate maximum sequence length.
11. Removed reviews longer than 16 words and applied post-padding so that all remaining review sequences contained a uniform length of 16 tokens.
12. Applied post-padding so that all review sequences contained a uniform length of 16 tokens.
13. Split the padded dataset into training, validation, and testing subsets using an 80/10/10 ratio.

An 80/10/10 split was selected because it is a commonly used approach in neural network modeling. The training set (80%) was used to train the LSTM model, the validation set (10%) was used to monitor performance during training and detect overfitting, and the testing set (10%) was reserved for final evaluation on unseen data. Stratified sampling was used to preserve the original sentiment distribution across all subsets. After preprocessing and outlier removal, the dataset contained 2,866 reviews, which were divided into 2,292 training reviews, 287 validation reviews, and 287 testing reviews.

```
#Export data
df.to_csv("cleaned_full_data.csv", index=False)

#Combine the split datasets SPECIFICALLY for exporting, do not alter the original

#Combine X_train and y_train together
train_df = pd.DataFrame(X_train)
train_df["sentiment"] = y_train
train_df.to_csv("train_dataset.csv", index=False)

#Combine X_val and y_val together
val_df = pd.DataFrame(X_val)
val_df["sentiment"] = y_val
val_df.to_csv("validation_dataset.csv", index=False)

#Combine X_test and y_test together
test_df = pd.DataFrame(X_test)
test_df["sentiment"] = y_test
test_df.to_csv("test_dataset.csv", index=False)
```

Output of model summary:

Layer (type)	Output Shape	Param #
embedding_5 (Embedding)	(None, 16, 128)	545,792
lstm_5 (LSTM)	(None, 64)	49,408
dropout_5 (Dropout)	(None, 64)	0
dense_10 (Dense)	(None, 32)	2,080
dense_11 (Dense)	(None, 1)	33

Total params: 597,313 (2.28 MB)

Trainable params: 597,313 (2.28 MB)

Non-trainable params: 0 (0.00 B)

Final Model Explanation:

The final neural network model consisted of five layers and contained 597,313 trainable parameters.

The first layer was an Embedding layer. The reviews had already been converted into integer tokens during tokenization, but neural networks cannot learn much from the raw token IDs themselves. The Embedding layer converted each token into a 128-dimensional vector representation. Similar words can develop similar vector representations during training, allowing the model to learn relationships between words. This layer contained 545,792 trainable parameters.

The second layer was an LSTM layer with 64 units. LSTM stands for Long Short-Term Memory and is a type of recurrent neural network designed for sequential data such as text. Unlike a traditional neural network, the LSTM processes words in order and can retain information from earlier words while evaluating later words. This is useful for sentiment analysis because phrases such as "not good" and "good" have different meanings even though they contain many of the same words. The LSTM layer contained 49,408 trainable parameters.

The third layer was a Dropout layer with a rate of 0.25. During training, this layer randomly ignores 25% of the outputs from the previous layer (Datacamp, n.d.). This helps reduce overfitting by preventing the model from relying too heavily on any one feature or pattern. The Dropout layer contained no trainable parameters.

The fourth layer was a Dense hidden layer with 32 neurons and a ReLU activation function. A Dense layer combines the information learned by previous layers and looks for higher-level patterns. The ReLU activation function returns positive values unchanged and converts negative values to zero (Aly, n.d., Datacamp, n.d.). This helps the network focus on useful signals while remaining computationally efficient. This layer contained 2,080 trainable parameters.

The fifth and final layer was a Dense output layer with one neuron and a sigmoid activation function. The sigmoid function converts the output into a value between 0 and 1, which can be interpreted as the probability that a review belongs to the positive sentiment class. Since this project involves binary classification (positive or negative sentiment), a single output neuron with sigmoid activation was appropriate. This layer contained 33 trainable parameters.

Overall, the model contained 597,313 trainable parameters, with most of the parameters located in the Embedding layer.

The model used several hyperparameters that were selected based on common practices for sentiment analysis with LSTM networks.

Activation functions:

The model used two activation functions: ReLU and Sigmoid.

The hidden dense layer used the ReLU activation function. After a neuron performs its calculations, ReLU keeps positive values and changes negative values to zero (Aly, n.d., Datacamp, n.d.). This helps the model focus on useful patterns and makes training faster.

The output layer used the Sigmoid activation function because this is a binary classification problem with only two possible outcomes: positive or negative sentiment. Sigmoid converts the final output into a value between 0 and 1 (Sewell & Datacamp, n.d.). For example, an output of 0.90 means the model predicts a 90% chance that the review is positive.

Number of Nodes per Layer:

The embedding layer used a vector size of 128 dimensions, as suggested by Dr. Sewell (Sewell, n.d.). This layer converts words into numerical representations that allow the model to learn relationships between words. A larger vector size can capture more information, but it also increases the number of trainable parameters and the risk of overfitting (Aly, n.d.). Since this was the initial model before tuning, 128 dimensions were used as a starting point.

The LSTM layer used 64 units. LSTM stands for Long Short-Term Memory and is a type of recurrent neural network designed to remember important information while reading text. This helps the model understand the meaning of an entire review instead of focusing only on individual words. For example, the model can remember words that appeared earlier in a sentence when deciding the overall sentiment.

The dense hidden layer used 32 nodes. This layer takes the information learned by the previous layers and combines it to help make the final sentiment prediction.

Loss Function:

Binary cross-entropy was used because the model only had to predict one of two classes: positive or negative sentiment. The loss function measures how close the model's predictions are to the correct answers. Lower loss values indicate better predictions.

Optimizer:

The Adam optimizer was used to update the model's weights during training (Datacamp, n.d.). After each batch of reviews, Adam adjusts the weights to help reduce prediction errors. Adam is commonly used because it usually learns quickly and performs well without requiring extensive tuning.

Stopping Criteria:

The model was trained for 20 epochs as a recommended starting parameter (Sewell, n.d.). An epoch represents one complete pass through all training reviews. Validation accuracy and validation loss were monitored throughout training to determine whether the model was still improving or beginning to overfit the training data.

Evaluation Metric:

Accuracy was selected as the evaluation metric because the goal of the model was to correctly classify reviews as positive or negative. Accuracy measures the percentage of reviews that were classified correctly. For example, an accuracy of 80% means the model correctly classified 80 out of 100 reviews.

Model Evaluation:

The model was trained for a maximum of 25 epochs. An epoch represents one complete pass through the training dataset. Rather than forcing the model to train through all 25 epochs, an EarlyStopping callback was initialized to monitor validation loss. As the further the model goes through epochs, the better it gets at memorizing the training data than predicting.

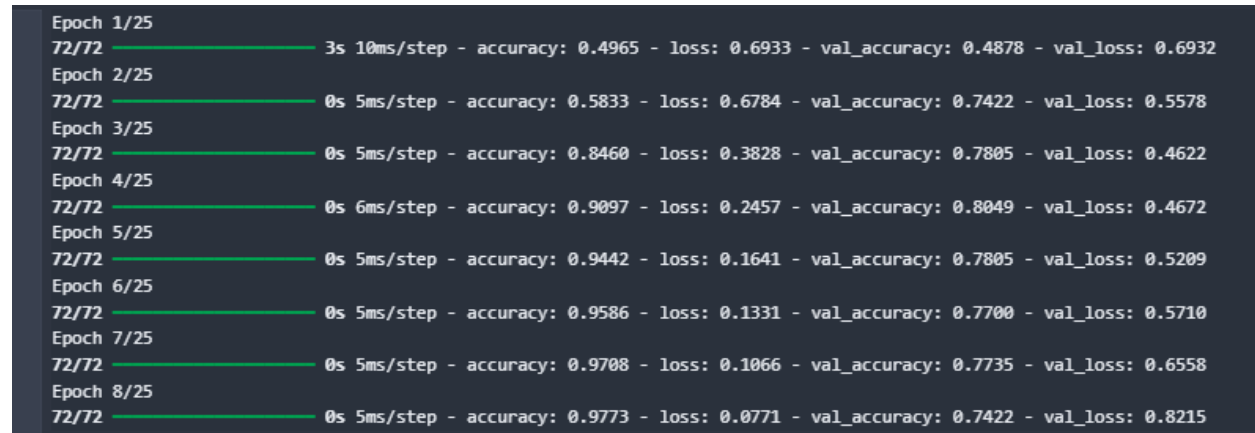
The stopping criteria were configured with a patience value of 5, meaning training would stop if the validation loss did not improve for 5 more epochs. The parameter `restore_best_weights=True` was also used so that the model would automatically return to the weights that produced the lowest validation loss.

Using early stopping helped reduce overfitting, as I noticed the best statistics occurred in the first 5 epochs in multiple models that were instantiated. During training, the model's accuracy continued to improve, while validating accuracy and validation loss eventually stopped improving. This indicated that the model was beginning to memorize the training data rather than learning patterns that generalized well to unseen reviews. By stopping training when validation no longer improved, the model maintained better generalization performance.

The final selected model achieved a validation accuracy of 80.49%, and a validation loss of 0.4622, while being much more stable as it does not dramatically drop in accuracy and validation compared to the other models built. When evaluating the model on the independent test set, the model achieved a test accuracy of 79.44%, and test loss of 0.5104. The small difference between validation and test set accuracy suggest that the stopping criteria were effective in limiting overfitting while preserving model performance.

Screenshot:

As shown in the screenshot, the model improved quickly during the first four epochs. Validation accuracy increased from 48.78% in Epoch 1 to 80.49% in Epoch 4. After that point, training accuracy continued to increase while validation accuracy began to decrease, and validation loss increased. This indicates that the model was starting to memorize the training data rather than improve its ability to classify new reviews. Early stopping helped prevent further overfitting by restoring the weights from the best-performing epoch.



Epoch 1/25	72/72	3s 10ms/step	- accuracy: 0.4965	- loss: 0.6933	- val_accuracy: 0.4878	- val_loss: 0.6932
Epoch 2/25	72/72	0s 5ms/step	- accuracy: 0.5833	- loss: 0.6784	- val_accuracy: 0.7422	- val_loss: 0.5578
Epoch 3/25	72/72	0s 5ms/step	- accuracy: 0.8460	- loss: 0.3828	- val_accuracy: 0.7805	- val_loss: 0.4622
Epoch 4/25	72/72	0s 6ms/step	- accuracy: 0.9097	- loss: 0.2457	- val_accuracy: 0.8049	- val_loss: 0.4672
Epoch 5/25	72/72	0s 5ms/step	- accuracy: 0.9442	- loss: 0.1641	- val_accuracy: 0.7805	- val_loss: 0.5209
Epoch 6/25	72/72	0s 5ms/step	- accuracy: 0.9586	- loss: 0.1331	- val_accuracy: 0.7700	- val_loss: 0.5710
Epoch 7/25	72/72	0s 5ms/step	- accuracy: 0.9708	- loss: 0.1066	- val_accuracy: 0.7735	- val_loss: 0.6558
Epoch 8/25	72/72	0s 5ms/step	- accuracy: 0.9773	- loss: 0.0771	- val_accuracy: 0.7422	- val_loss: 0.8215

Fitness of the Model:

The final model was a better fit for the data than the initial model. Model 1 contained 597,313 trainable parameters and achieved very high training accuracy. However, validation accuracy decreased over time while validation loss increased. This suggested that the model was beginning to memorize the training reviews instead of learning patterns that would work well on new reviews.

Several changes were made to improve the model and reduce overfitting. First, the embedding size was reduced from initial 128 as Dr. Sewells suggestion to 32. This made the model much smaller and reduced the number of parameters it needed to learn. The LSTM layer was also reduced from 64 units to 48 units. Since the reviews in the dataset were relatively short(max 16 words), a smaller LSTM was still able to understand the sentence context while reducing model complexity.

Another change was adding a second dense layer. In Model 1, the LSTM output only passed through one dense layer before making a prediction. In the final model, the output passed through two dense layers with 64 and 48 nodes. This gave the model an extra step to process the information learned from the reviews before deciding whether the sentiment was positive or negative. Testing showed that this change improved validation accuracy compared to earlier models.

The dropout rate was reduced from 0.25 to 0.01. Since the model had already been made much smaller by reducing the embedding size and LSTM units, less dropout was needed. This allowed the model to keep more information during training while still providing some protection against overfitting.

Early stopping was also added to monitor validation loss. If the model stopped improving on the validation data, training would stop automatically and restore the best-performing weights.

These changes reduced the number of trainable parameters from 597,313 in Model 1 to 158,305 in the final model, a reduction of about 73%. Despite being much smaller, the final model achieved a validation accuracy of 80.49% and a test accuracy of 79.44%, suggesting that it generalized better to unseen reviews than the original model.

Model 1 summary:

Layer (type)	Output Shape	Param #
embedding (Embedding)	(None, 16, 128)	545,792
lstm (LSTM)	(None, 64)	49,408
dropout (Dropout)	(None, 64)	0
dense (Dense)	(None, 32)	2,080
dense_1 (Dense)	(None, 1)	33

Final Model summary:

Layer (type)	Output Shape	Param #
embedding_3 (Embedding)	(None, 16, 32)	136,448
lstm_3 (LSTM)	(None, 48)	15,552
dense_6 (Dense)	(None, 64)	3,136
dense_7 (Dense)	(None, 48)	3,120
dropout_3 (Dropout)	(None, 48)	0
dense_8 (Dense)	(None, 1)	49

Model 1 epoch:

```
Epoch 1/20
72/72 ————— 2s 7ms/step - accuracy: 0.5039 - loss: 0.6942 - val_accuracy: 0.4983 - val_loss: 0.6910
Epoch 2/20
72/72 ————— 0s 6ms/step - accuracy: 0.7749 - loss: 0.4872 - val_accuracy: 0.8084 - val_loss: 0.4412
Epoch 3/20
72/72 ————— 1s 5ms/step - accuracy: 0.9215 - loss: 0.2181 - val_accuracy: 0.7840 - val_loss: 0.5433
Epoch 4/20
72/72 ————— 0s 4ms/step - accuracy: 0.9555 - loss: 0.1478 - val_accuracy: 0.7631 - val_loss: 0.7225
Epoch 5/20
72/72 ————— 0s 4ms/step - accuracy: 0.9668 - loss: 0.1080 - val_accuracy: 0.7596 - val_loss: 0.7236
Epoch 6/20
72/72 ————— 0s 5ms/step - accuracy: 0.9734 - loss: 0.0892 - val_accuracy: 0.7526 - val_loss: 0.7060
Epoch 7/20
72/72 ————— 0s 5ms/step - accuracy: 0.9847 - loss: 0.0623 - val_accuracy: 0.7456 - val_loss: 0.9456
Epoch 8/20
72/72 ————— 0s 5ms/step - accuracy: 0.9860 - loss: 0.0536 - val_accuracy: 0.7526 - val_loss: 0.9080
Epoch 9/20
72/72 ————— 0s 5ms/step - accuracy: 0.9869 - loss: 0.0522 - val_accuracy: 0.7352 - val_loss: 1.1097
Epoch 10/20
72/72 ————— 0s 4ms/step - accuracy: 0.9865 - loss: 0.0560 - val_accuracy: 0.7317 - val_loss: 1.2255
```

Final Model epoch:

```
Epoch 1/25
72/72 ————— 3s 10ms/step - accuracy: 0.4965 - loss: 0.6933 - val_accuracy: 0.4878 - val_loss: 0.6932
Epoch 2/25
72/72 ————— 0s 5ms/step - accuracy: 0.5833 - loss: 0.6784 - val_accuracy: 0.7422 - val_loss: 0.5578
Epoch 3/25
72/72 ————— 0s 5ms/step - accuracy: 0.8460 - loss: 0.3828 - val_accuracy: 0.7805 - val_loss: 0.4622
Epoch 4/25
72/72 ————— 0s 6ms/step - accuracy: 0.9097 - loss: 0.2457 - val_accuracy: 0.8049 - val_loss: 0.4672
Epoch 5/25
72/72 ————— 0s 5ms/step - accuracy: 0.9442 - loss: 0.1641 - val_accuracy: 0.7805 - val_loss: 0.5209
Epoch 6/25
72/72 ————— 0s 5ms/step - accuracy: 0.9586 - loss: 0.1331 - val_accuracy: 0.7700 - val_loss: 0.5710
Epoch 7/25
72/72 ————— 0s 5ms/step - accuracy: 0.9708 - loss: 0.1066 - val_accuracy: 0.7735 - val_loss: 0.6558
Epoch 8/25
72/72 ————— 0s 5ms/step - accuracy: 0.9773 - loss: 0.0771 - val_accuracy: 0.7422 - val_loss: 0.8215
```

At a small sacrifice in validation accuracy, the final model required significantly fewer trainable parameters while still achieving similar performance. The number of trainable parameters was reduced from 597,313 in Model 1 to 158,305 in the final model. Despite being much smaller, the final model achieved a validation accuracy of 80.49% and a test accuracy of 79.44%, suggesting that it learned the overall sentiment patterns in the reviews without relying on a large number of parameters.

Results of model on test data:

```
[315]: #Now do it on test set

test_loss, test_accuracy = model4.evaluate(X_test, y_test)

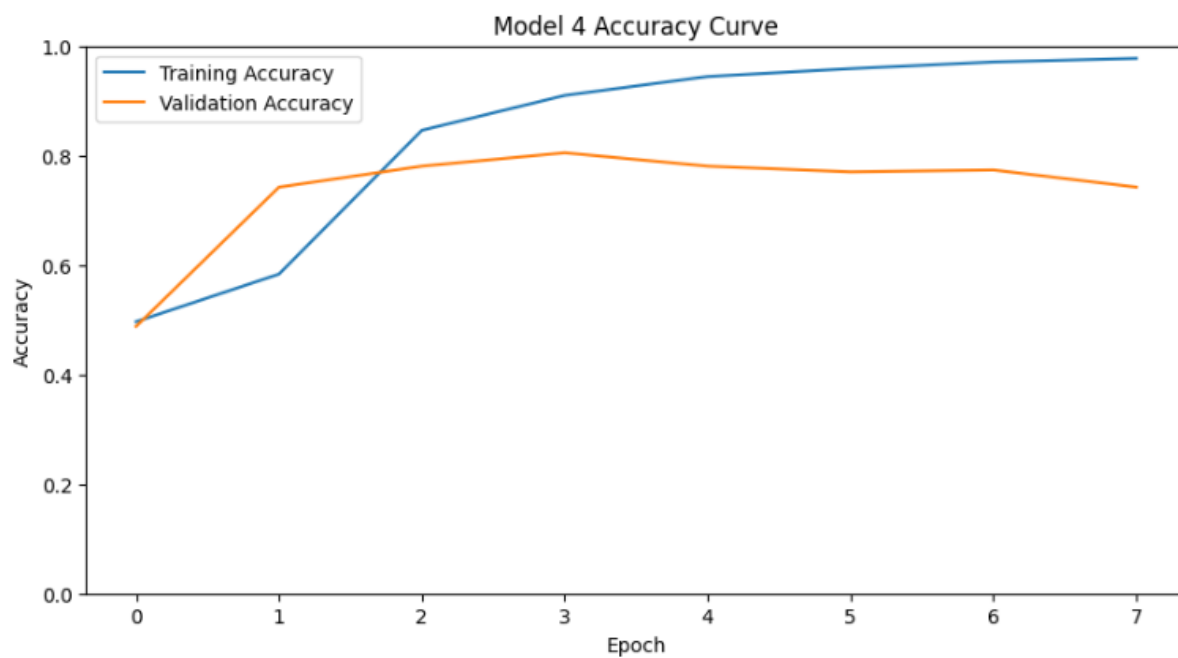
print("Test Accuracy:", test_accuracy)
print("Test Loss:", test_loss)

9/9 ————— 0s 3ms/step - accuracy: 0.7944 - loss: 0.5104
Test Accuracy: 0.7944250702857971
Test Loss: 0.510386049747467
```

Final Model plots:

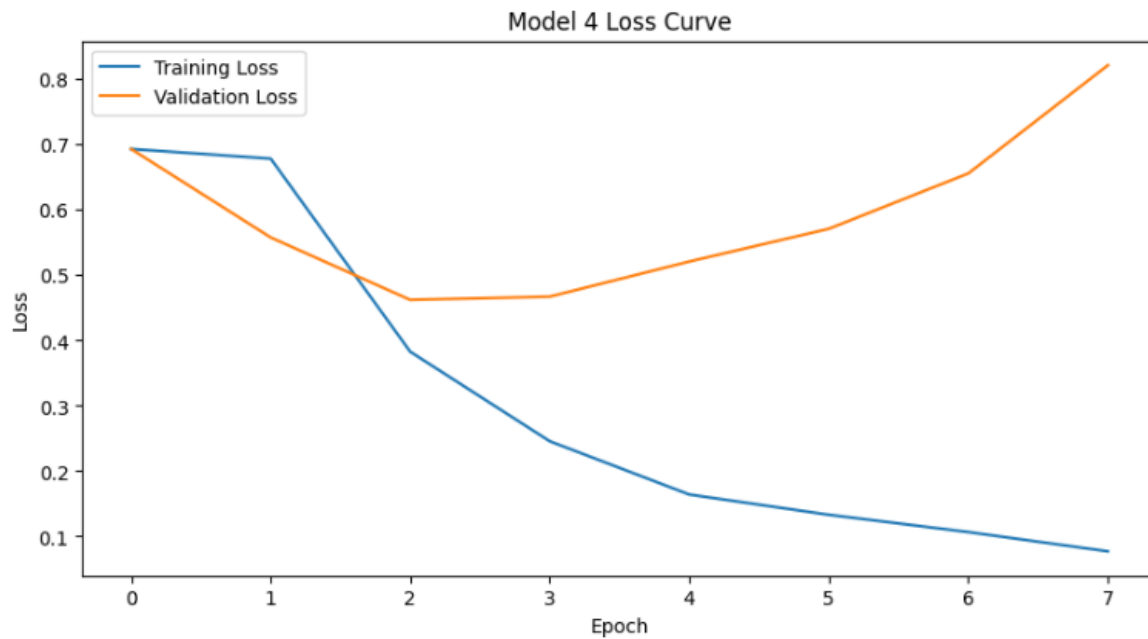
Accuracy Curve:

The plot shows that both training and validation accuracy improved rapidly during the first few epochs. Validation accuracy reached its highest value of approximately 80% around epoch 4 before gradually declining. Training accuracy continued to increase throughout training, suggesting that the model was continuing to learn patterns from the training data and beginning to overfit.



Loss Curve:

The loss curve shows that training loss steadily decreased throughout training, indicating that the model became better at fitting the training data. Validation loss decreased during the first few epochs and reached its lowest value around epoch 3 before increasing afterwards. This increase in validation loss suggests that overfitting began to occur after the model had learned the most useful patterns from the validation data.



Predictive Accuracy:

Accuracy was chosen as the evaluation metric because the goal of the model was to correctly classify reviews as either positive or negative. When the final model was tested on data it had never seen before, it achieved a test accuracy of 79.44%. This means the model correctly classified about 8 out of every 10 reviews.

The confusion matrix provides a more detailed look at the model's predictions. The model correctly identified 108 negative reviews and 120 positive reviews. It incorrectly classified 36 negative reviews as positive and 23 positive reviews as negative. In total, the model correctly classified 228 out of 287 reviews.

The model achieved a precision of 76.92%, meaning that when it predicted a review was positive, it was correct about 77% of the time. The model achieved a recall of 83.92%, meaning it successfully found most of the positive reviews in the dataset. The specificity of 75.00% indicates that the model correctly identified about three out of every four negative reviews.

Overall, the results show that the model was able to learn sentiment patterns from the reviews and make accurate predictions on new data. These results suggest that the model was able to generalize well to new data and can reliably identify sentiment in customer reviews.

Accuracy on test set:

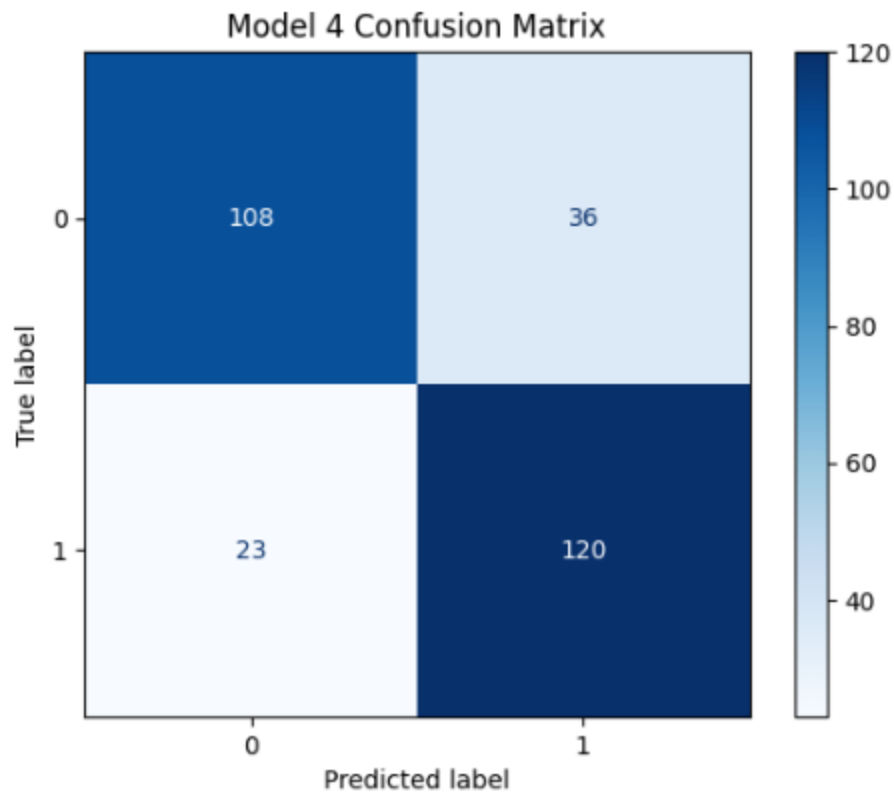
79.44% accuracy on the holdout data

```
#Now do it on test set

test_loss, test_accuracy = model4.evaluate(X_test, y_test)

print("Test Accuracy:", test_accuracy)
print("Test Loss:", test_loss)

9/9 ————— 0s 14ms/step - accuracy: 0.7944 - loss: 0.5104
Test Accuracy: 0.7944250702857971
Test Loss: 0.510386049747467
```



Accuracy: 79.44250871080139
Precision: 76.92307692307693
Recall: 83.91608391608392
Specificity: 75.0

Functionality of Neural Network and Architecture:

The final neural network performed well on the sentiment analysis task and achieved a test accuracy of 79.44%. Dr. Sewell notes that a model with at least 70% accuracy is generally considered adequate for this project, so the final model exceeded that benchmark.

During development, several model architectures were tested. The original model used a larger embedding layer and LSTM layer, but it showed signs of overfitting. To improve performance, the embedding size was reduced from 128 to 32, the LSTM layer was reduced from 64 to 48 units, and an additional dense layer was added. These changes reduced the number of trainable parameters while improving validation accuracy.

Dropout layers were also used to help reduce overfitting and improve the model's ability to generalize to unseen reviews. Overall, the final architecture provided a good balance between accuracy, model complexity, and predictive performance.

Business Recommendation:

The recommended course of action is to use the final model as an automated sentiment analysis tool for customer reviews. The model achieved a test accuracy of 79.44%, suggesting that it can reliably classify reviews as positive or negative without requiring manual review.

One advantage of this model is that it was trained using review data from Amazon, Yelp, and IMDb. Because these sources contain product reviews, restaurant reviews, and movie reviews, the model was exposed to a wider variety of writing styles and sentiment expressions. This may help the model generalize better than a model trained on a single type of review.

Future improvements could include training on a larger dataset, testing additional neural network architectures, or using pretrained word embeddings. These changes may improve predictive accuracy and reduce classification errors. Overall, the results suggest that the final model provides a solid foundation for sentiment analysis across multiple review domains.